

Intro Hugging Face and Python

Dirk Wulff & Zak Hussain



MAX PLANCK INSTITUTE
FOR HUMAN DEVELOPMENT



OUR SOFTWARE STACK



+



+



OUR SOFTWARE STACK



+



+



WHY IS HUGGING FACE?



WHY IS HUGGING FACE?



Traditional language modelling pipeline:

WHY IS HUGGING FACE?



Traditional language modelling pipeline:

1. Find out the model architecture

WHY IS HUGGING FACE?



Traditional language modelling pipeline:

1. Find out the model architecture
2. Implement the model architecture in code with deep learning frameworks (e.g PyTorch/Tensorflow).

1. DEEP LEARNING LIBRARIES CAN BE DIFFICULT

1. DEEP LEARNING LIBRARIES CAN BE DIFFICULT

I  KING HATE TENSORFLOW #53549

✓ Closed

ghost opened this issue 9 hours ago · 2 comments

WHY HUGGING FACE?

Traditional language modelling pipeline:

1. Find out the model architecture
2. Implement the model architecture in code with deep learning libraries (e.g PyTorch/Tensorflow).

WHY HUGGING FACE?

Traditional language modelling pipeline:

1. Find out the model architecture
2. Implement the model architecture in code with deep learning libraries (e.g PyTorch/Tensorflow).
3. Load the pretrained weights (if available) from a server.
4. Process the inputs (using the correct tokenizer for the model)
5. Implement data loaders
6. Define a loss function
7. Stick a task-specific “head” on the model

HUGGING FACE PIPELINES

Import pipeline

```
from transformers import pipeline
```

Initialise
pipeline

```
pipe = pipeline('text-generation', model='gpt2')
```

Load model
input

```
prompt = """
```

```
Once upon a time in a land far far away, there was a young prince named  
John. He was known for his bravery and courage. One day, he decided to go on  
an adventure to explore the unknown lands.
```

```
"""
```

Feed input
the model

```
output = pipe(prompt, max_length=100)
```

```
print(output)
```

HUGGING FACE

 **Hugging Face**

Models

Datasets

Spaces

Community

Docs

Enterprise

Pricing

Main

Tasks

Libraries

Languages

Licenses

Other

Text Generation

Any-to-Any

Image-Text-to-Text

Image-to-Text

Image-to-Image

Text-to-Image

Text-to-Video

Text-to-Speech

+ 42

Parameters

< 1B

6B

12B

32B

128B

> 500B

Libraries

PyTorch

TensorFlow

JAX

Transformers

Diffusers

sentence-transformers

Safetensors

ONNX

GGUF

Transformers.js

MLX

+ 41

Apps

vLLM

TGI

llama.cpp

MLX LM

LM Studio

Ollama

Jan

+ 13

Inference Providers

Groq

Novita

Nebius AI

Cerebras

SambaNova

Nscale

fal

Hyperbolic

+ 11

Models 2,256,270

Filter by name

Full-text search

Inference Available

Sort: Trending

Tongyi-MAI/Z-Image-Turbo

Text-to-Image · Updated about 23 hours ago · ± 111k · ± 1.91k

deepseek-ai/DeepSeek-V3.2

Text Generation · ± 685B · Updated 2 days ago · ± 5.01k · ± 626

deepseek-ai/DeepSeek-Math-V2

Text Generation · ± 685B · Updated 6 days ago · ± 7.21k · ± 616

deepseek-ai/DeepSeek-V3.2-Special

Text Generation · ± 685B · Updated 2 days ago · ± 1.87k · ± 439

black-forest-labs/FLUX.2-dev

Image-to-Image · Updated 6 days ago · ± 185k · ± 850

Comfy-Org/z_image_turbo

Updated 6 days ago · ± 1.5M · ± 339

tencent/HunyuanOCR

Image-Text-to-Text · ± 1.0B · Updated 1 day ago · ± 226k · ± 614

nvidia/Orchestrator-8B

Text Generation · ± 8B · Updated 1 day ago · ± 1.04k · ± 269

facebook/sam3

Mask Generation · ± 0.9B · Updated 13 days ago · ± 327k · ± 858

microsoft/Fara-7B

Image-Text-to-Text · ± 8B · Updated 2 days ago · ± 25.3k · ± 388

apple/starflow

Updated 1 day ago · ± 172

alibaba-pai/Z-Image-Turbo-Fun-Controlnet-Union

Updated 1 day ago · ± 177

jayn7/Z-Image-Turbo-GGUF

Text-to-Image · ± 6B · Updated 5 days ago · ± 70k · ± 154

PrimeIntellect/INTELLECT-3

Text Generation · ± 107B · Updated 6 days ago · ± 2.75k · ± 170

mistralai/Mistral-large-3-675B-Instruct-2512

Updated about 19 hours ago · ± 112 · ± 113

AIDC-AI/Ovis-Image-7B

Text-to-Image · Updated about 8 hours ago · ± 1.26k · ± 139

tencent/HunyuanVideo-1.5

Text-to-Video · Updated 1 day ago · ± 2.63k · ± 791

stepfun-ai/Step-Audio-R1

Audio-Text-to-Text · ± 33B · Updated 1 day ago · ± 315 · ± 110

TSB/Z-Image-Turbo-FP8

Text-to-Image · Updated 6 days ago · ± 110k · ± 91

Supertone/supertonic

Text-to-Speech · Updated 11 days ago · ± 14.4k · ± 383

arcee-ai/Trinity-Mini

Text Generation · ± 26B · Updated 1 day ago · ± 240 · ± 87

salakash/SamKash-Tolstoy

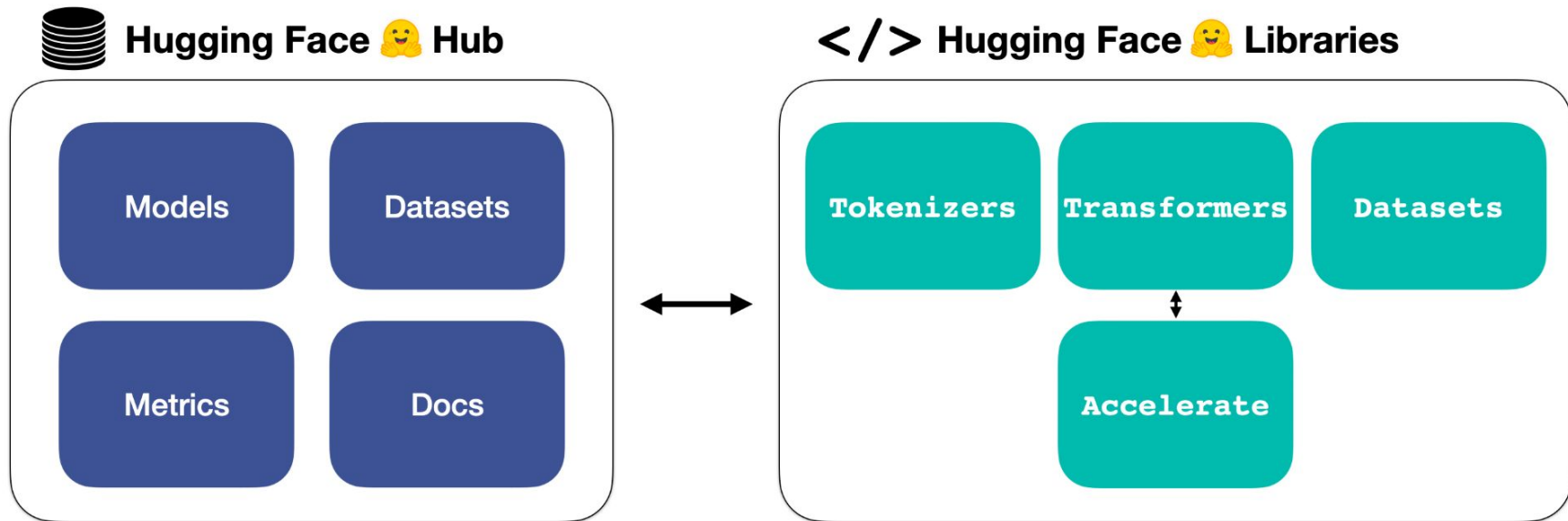
Text Generation · Updated 6 days ago · ± 7.88k · ± 469

unsloth/DeepSeek-V3-00B-A3B-TextInstruct-GGUF

unsloth/Mistral-7B-14B-TextInstruct-2512

LLM4BeSci Zurich 2025

THE HUGGING FACE ECOSYSTEM



HUGGING FACE DOCUMENTATION

Documentations

 Search across all docs

• Hub

Host Git-based models, datasets and Spaces on the Hugging Face Hub.

• Hub Python Library

Client library for the HF Hub: manage repositories from your Python runtime.

• Inference API

Use more than 50k models through our public inference API, with scalability built-in.

• Transformers

State-of-the-art ML for Pytorch, TensorFlow, and JAX.

• Datasets

Access and share datasets for computer vision, audio, and NLP tasks.

• Huggingface.js

A collection of JS libraries to interact with Hugging Face, with TS types included.

• Inference Endpoints

Easily deploy your model to production on dedicated, fully managed infrastructure.

• Diffusers

State-of-the-art diffusion models for image and audio generation in PyTorch.

• Gradio

Build machine learning demos and other web apps, in just a few lines of Python.

• Transformers.js

Community library to run pretrained models from Transformers in your browser.

• PEFT

Parameter efficient finetuning methods for large models

OUR SOFTWARE STACK



+



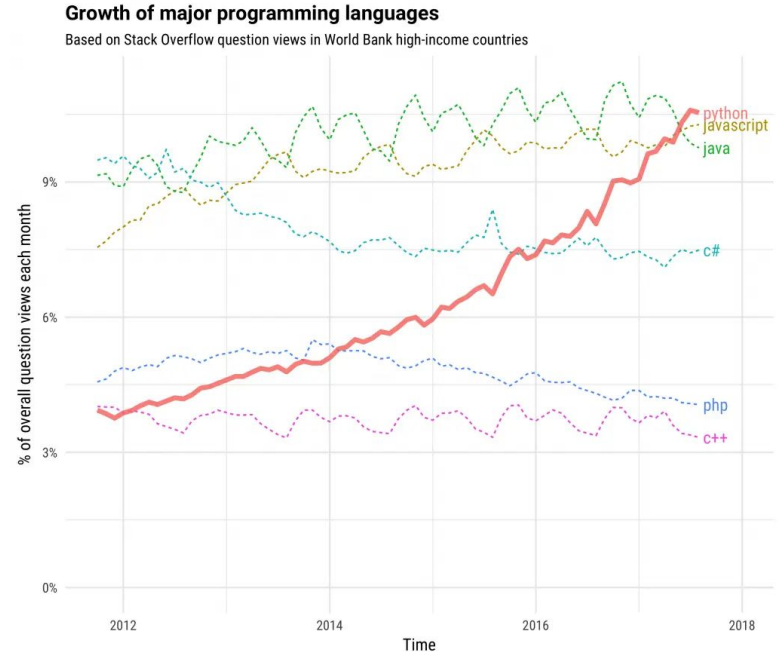
+





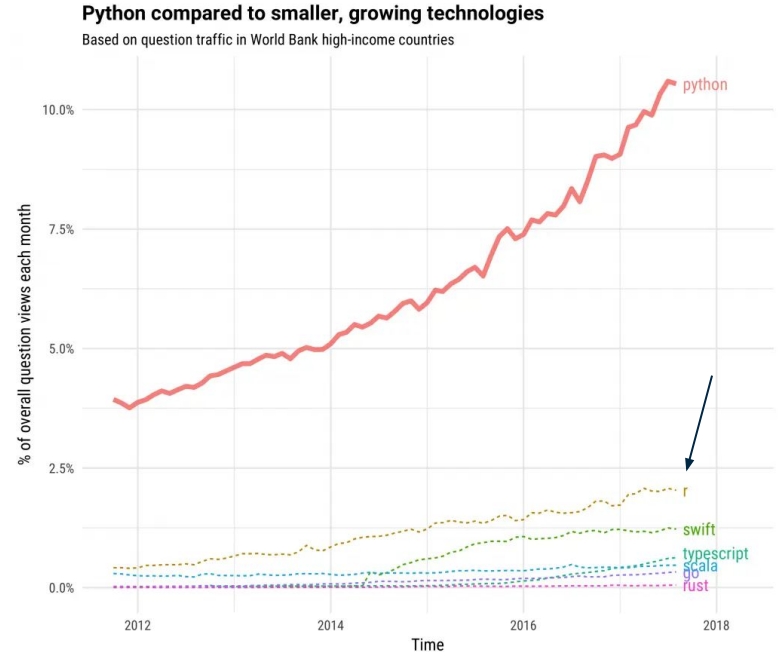


- A **high-level** programming language known for its simplicity and readability.
- Used in various domains such as web development, data analysis, **artificial intelligence**, and scientific computing.





- A **high-level** programming language known for its simplicity and readability.
- Used in various domains such as web development, data analysis, **artificial intelligence**, and scientific computing.



OUR SOFTWARE STACK



+



+



PYTHON  + Google Colab 

PYTHON + Google Colab

 day_1.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

+ Code + Text

RAM
Disk

 [6]

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

 Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the `sentence-transformers` package. We will use the following three sentences, stored as a list of strings, as input to the model:

 [7]

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

We will use the `all-MiniLM-L6-v2` model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.



```
# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

`/usr/local/lib/python3.10/dist-packages/huggingface_hub/file_download.py:1132: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resu`

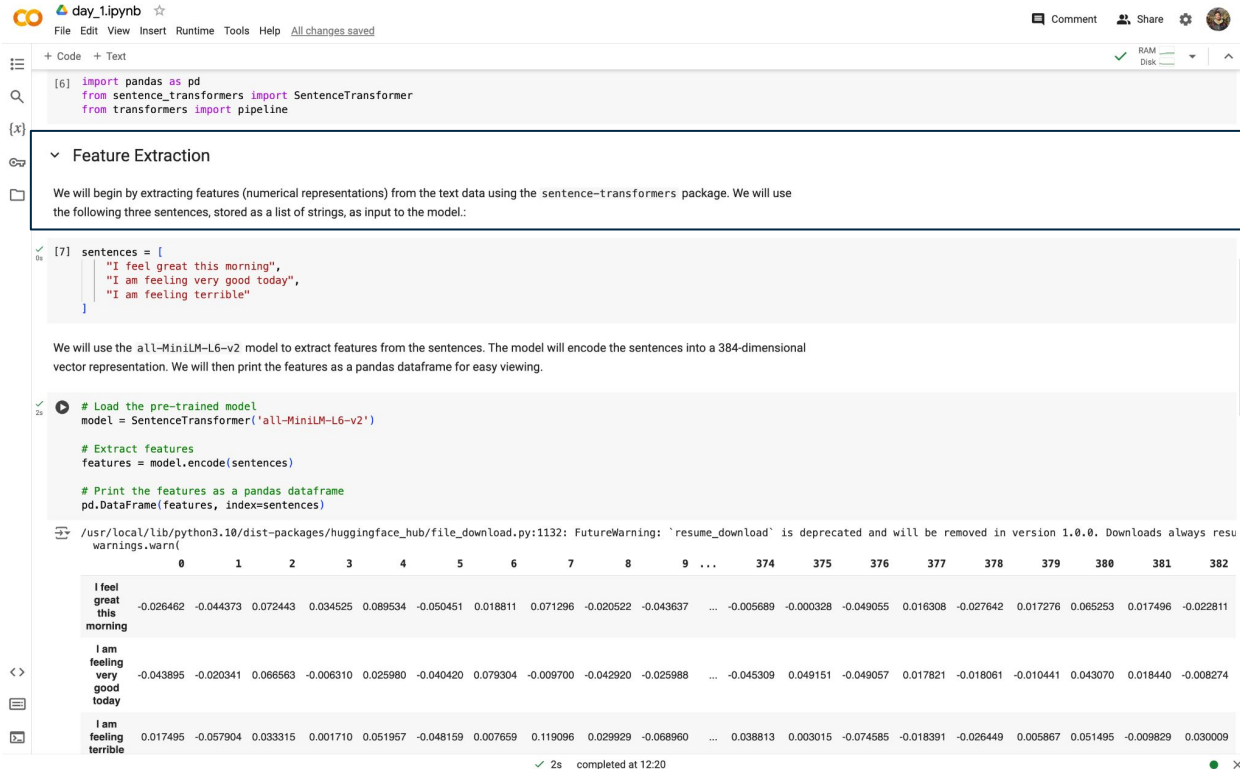
`warnings.warn(`

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.068563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

✓ 2s completed at 12:20

PYTHON + Google Colab

1. Markdown



The screenshot shows a Google Colab notebook titled "day_1.ipynb". The notebook contains a code cell with the following Python code:

```
[6] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

Below the code cell is a markdown cell titled "Feature Extraction". The text in the markdown cell reads:

We will begin by extracting features (numerical representations) from the text data using the `sentence-transformers` package. We will use the following three sentences, stored as a list of strings, as input to the model:

```
[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

Below the markdown cell is another markdown cell that reads:

We will use the `all-MiniLM-L6-v2` model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

Below the markdown cell is a code cell with the following Python code:

```
# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

Below the code cell is a warning message:

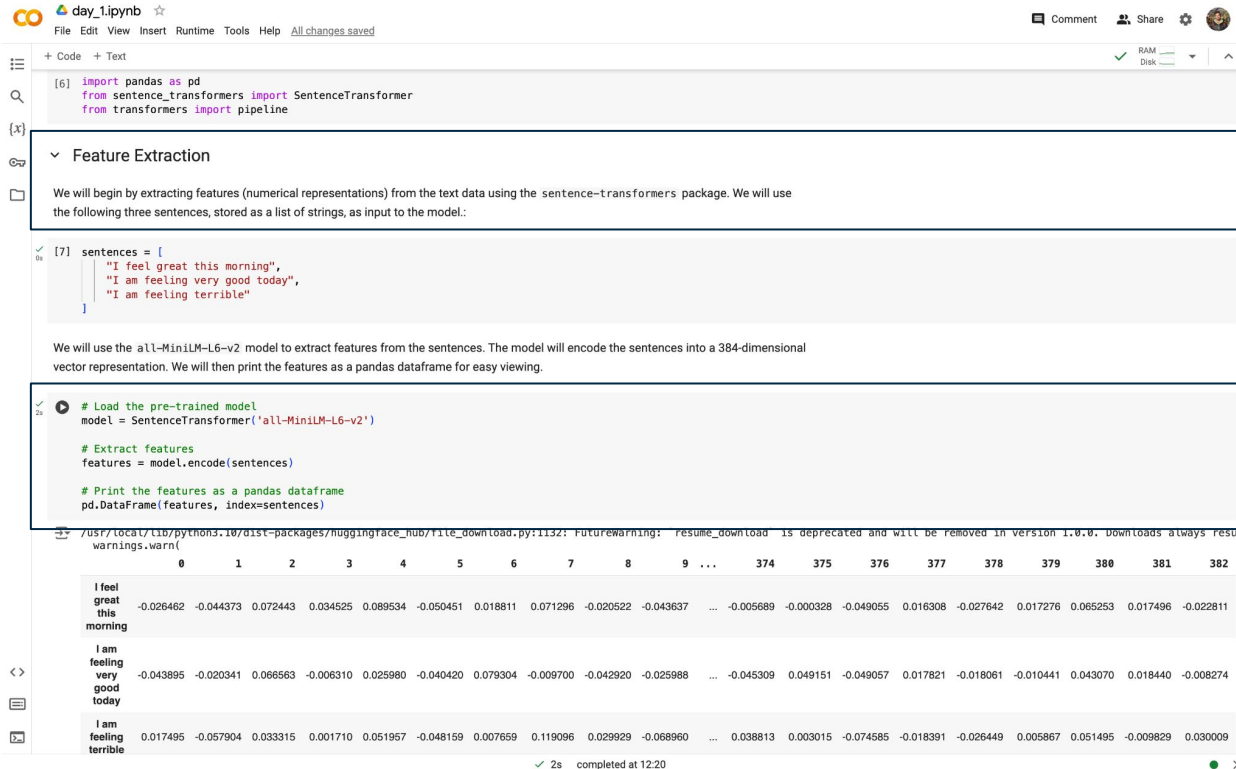
```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/file_download.py:1132: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resu
warnings.warn(
```

Below the warning message is a pandas dataframe showing the features for the three sentences:

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

PYTHON + Google Colab

1. Markdown



The screenshot shows a Google Colab notebook titled "day_1.ipynb". It contains a code cell with imports for pandas, sentence-transformers, and transformers. Below the code is a markdown cell titled "Feature Extraction" which explains the goal of extracting numerical features from text using the sentence-transformers package. It lists three sentences as input. Another code cell shows the sentences being defined as a list of strings. A third markdown cell explains that the all-MiniLM-L6-v2 model will be used to extract features, encoding sentences into a 384-dimensional vector representation. A final code cell shows the model being loaded, features being extracted, and the results being printed as a pandas DataFrame. The output of the code cell is a DataFrame with 384 columns and 3 rows, showing the numerical feature vectors for each sentence.

```
[6] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model:

```
[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

```
26 # Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

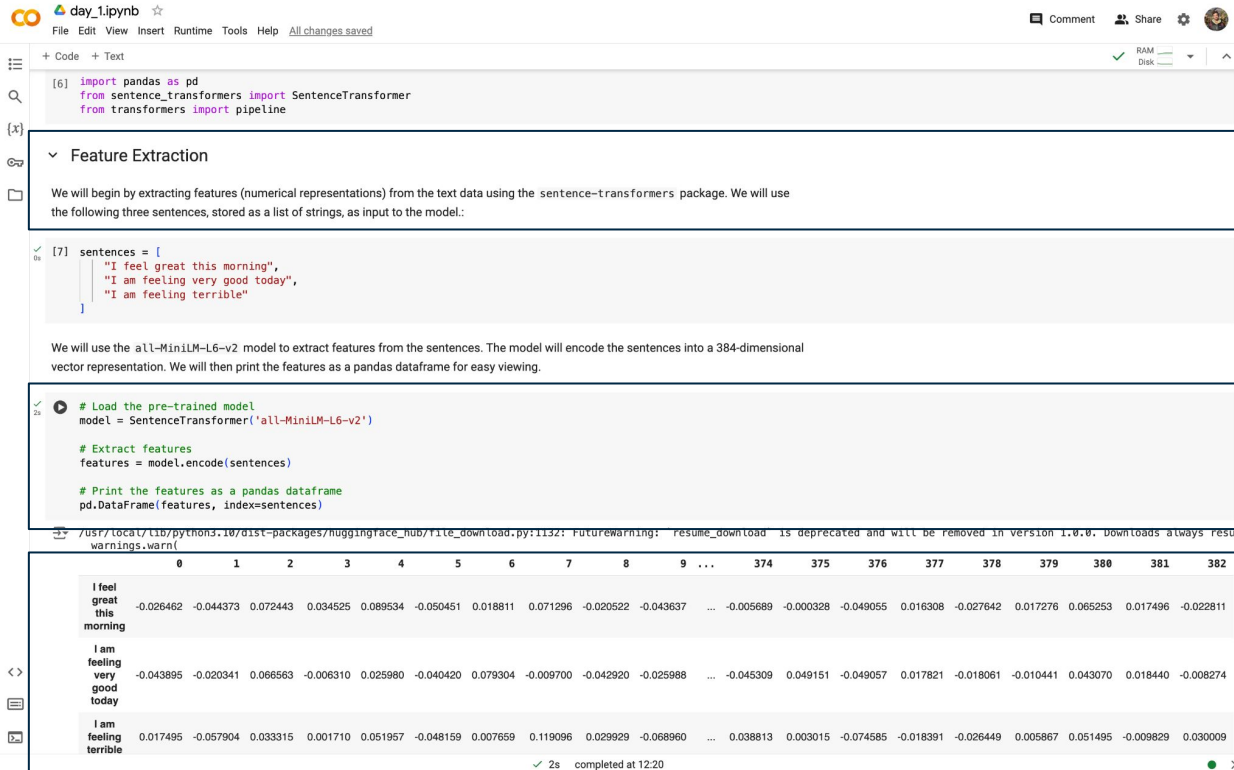
FutureWarning: resume_download is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. To avoid this warning, you can set the variable resume_download=False.

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

2. Code

PYTHON + Google Colab

1. Markdown



The screenshot shows a Google Colab notebook titled "day_1.ipynb". It contains three cells:

- Code Cell [6]:** Imports pandas as pd, SentenceTransformer from sentence_transformers, and pipeline from transformers.
- Markdown Cell:** Titled "Feature Extraction", it explains that numerical representations will be extracted from text data using the sentence-transformers package. It lists three sentences as input: "I feel great this morning", "I am feeling very good today", and "I am feeling terrible".
- Code Cell [7]:** Defines a list of sentences, loads the all-MiniLM-L6-v2 model, extracts features, and prints them as a pandas dataframe.

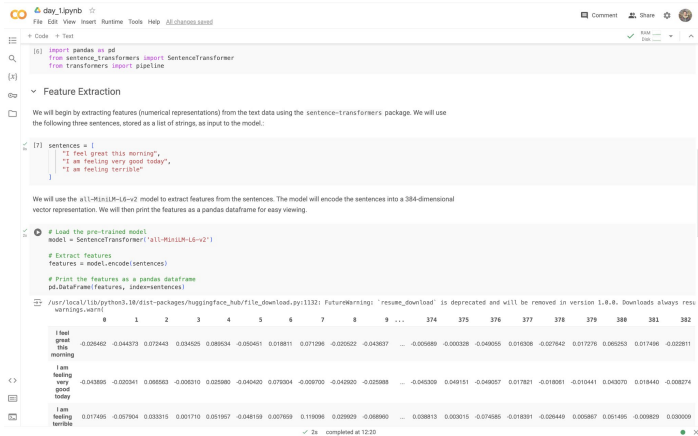
The output of the code cell shows a pandas dataframe with 384 columns and 3 rows of numerical features.

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

2. Code

3. Printouts

PYTHON + Google Colab



```
day_1pythb
File Edit View Insert Runtime Tools Help 88.00000000000000
+ Code + Text
[5] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline

Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model.

[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)

/usr/local/lib/python3.10/dist-packages/huggingface_hub/file_download.py:1132: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible.
FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible.

I feel great this morning
-0.026462 -0.044373 0.072943 0.034026 0.089504 -0.050461 0.018811 0.071296 -0.020522 -0.043037 ... -0.005889 -0.000328 -0.049005 0.016308 -0.027642 0.017276 0.060253 -0.017406 -0.002811

I am feeling very good today
-0.043886 -0.020341 -0.080903 -0.006210 0.025890 -0.040403 0.077604 -0.008700 -0.040262 -0.025088 ... -0.040309 0.068151 -0.048057 0.017821 -0.018061 -0.010441 0.040270 0.018440 -0.008274

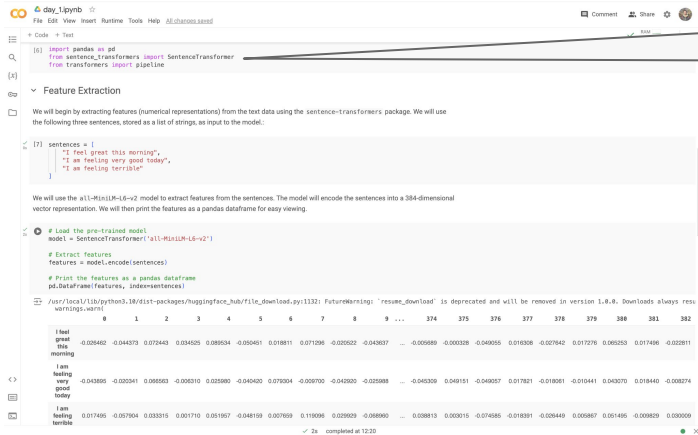
I am feeling terrible
0.017495 -0.057604 0.033315 0.001710 0.051957 -0.048159 0.007659 0.119006 0.029929 -0.088900 ... 0.008813 0.003015 -0.074585 -0.018391 -0.026410 0.005867 0.051405 -0.006829 0.030009

2s completed at 12:20
```

PYTHON + Google Colab

package imports

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```



```
6 day_1.pytnb
File Edit View Insert Runtime Tools Help
+ Code + Text
[5] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline

Feature Extraction
We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model.

[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072943	0.034026	0.089034	-0.050461	0.018811	0.071296	-0.020522	-0.043037	...	-0.005889	-0.000328	-0.049005	0.016308	-0.027642	0.017276	0.060253	0.017406	-0.020811
I am feeling very good today	-0.043886	-0.020341	0.080903	-0.006210	0.025890	-0.040403	0.076004	-0.008700	-0.040262	-0.020588	...	-0.046309	0.068151	-0.048037	0.017821	-0.018061	-0.010441	0.040270	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051857	-0.048159	0.007659	0.119006	0.022929	-0.088900	...	0.008813	0.003015	-0.074585	-0.018391	-0.026440	0.005867	-0.051405	-0.008629	0.030008

2s completed at 12:20

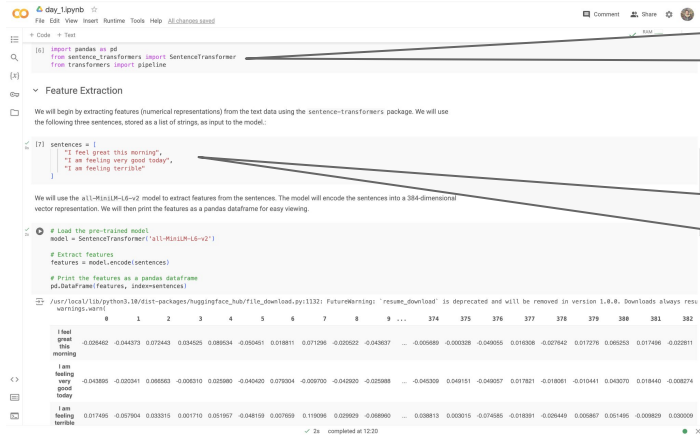
PYTHON + Google Colab

package imports

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

variable assignment, lists, strings

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```



```
day_1.py:nb
File Edit View Insert Runtime Tools Help 88.48MB/100MB
+ Code + Text
[5] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline

Feature Extraction
We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model.

[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)

/usr/local/lib/python3.10/dist-packages/huggingface_hub/file_download.py:1132: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible.
>>
I feel great this morning
0 1 2 3 4 5 6 7 8 ... 374 375 376 377 378 379 380 381 382
-0.036462 -0.043373 0.072943 0.034326 0.089304 -0.050461 0.018811 0.071396 -0.020522 -0.043037 ... -0.005889 -0.000328 -0.049005 0.016306 -0.027642 0.012726 0.060253 0.017406 -0.020811

I am feeling very good today
-0.043886 -0.020341 0.088953 -0.006310 0.025890 -0.040430 0.076204 -0.008700 -0.042620 -0.020588 ... -0.043309 0.068151 -0.048057 0.017821 -0.018061 -0.010441 0.040270 0.018440 -0.008274

I am feeling terrible
0.017495 -0.027604 0.033315 0.001710 0.051867 -0.048159 0.007659 0.119306 0.022929 -0.088900 ... 0.008813 0.003075 -0.074585 -0.018391 -0.026440 0.005867 0.051405 -0.008629 0.030009

2x completed at 12:20
```

PYTHON + Google Colab

package imports

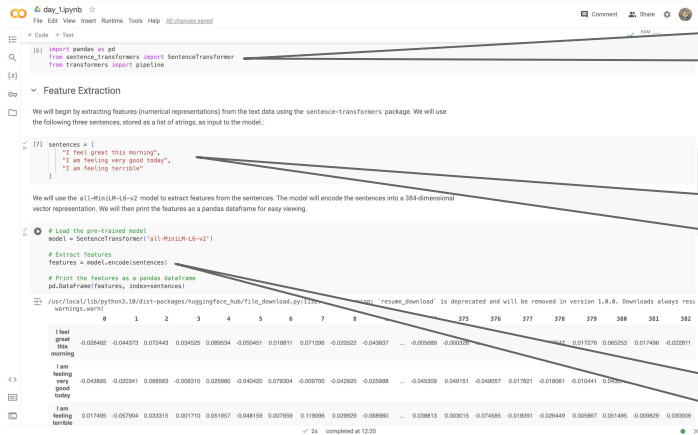
```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

variable assignment, lists, strings

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

dot notation, methods, attributes

```
# Extract features
features = model.encode(sentences)
```



The screenshot shows a Google Colab notebook with the following content:

```
File Edit View Insert Runtime Tools Help
+ Code + Text
[1] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model:

```
[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

```
# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

The output shows a pandas DataFrame with 3 rows and 384 columns. The first row corresponds to "I feel great this morning", the second to "I am feeling very good today", and the third to "I am feeling terrible". The values are numerical representations of the sentences.

PYTHON + Google Colab

package imports

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

variable assignment, lists, strings

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

dot notation, methods, attributes

```
# Extract features
features = model.encode(sentences)
```

printing

```
# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

	0	1	2	3	4	5	6
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304
I am feeling terrible	-0.017495	-0.027604	0.033203	0.048159	0.007659	0.119006	0.022929

OUR SOFTWARE STACK



+



+



Time to install our stack...



MAX PLANCK INSTITUTE
FOR HUMAN DEVELOPMENT



LLM4BeSci_Zurich2025Public

PinWatch 0Fork 0Star 0

main 1 Branch 0 Tags

Go to fileAdd fileCode

About

 Zak-Hussain	populate	155e7c9 · 2 hours ago	2 Commits
day_1	populate	2 hours ago	
day_2	populate	2 hours ago	
.gitignore	Initial commit	4 hours ago	
LICENSE.txt	populate	2 hours ago	
README.md	populate	2 hours ago	
notes.txt	populate	2 hours ago	

The course introduces the use of open-source large language models (LLMs) from the Hugging Face ecosystem for research in the behavioral and social sciences.

ReadmeView licenseActivity0 stars0 watching0 forks

READMELicense

LLM4BeSci at Uni Zurich, 2025

The course introduces the use of open-source large language models (LLMs) from the Hugging Face ecosystem for research in the behavioral and social sciences.

Materials by [Dirk Wulff](#) and [Zak Hussain](#). Taught in-person by Zak Hussain.

Schedule

Day 1

09:30 AM - 09:45 AM: Welcome & Intro

09:45 AM - 10:30 AM: Talk: Intro to LLMs

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Languages

Jupyter Notebook 100.0%